

ML/AI এর জন্য Complete Pandas Course

Table of Contents

1. [Pandas Introduction & Installation](#)
 2. [Data Structures \(Series & DataFrame\)](#)
 3. [Data Loading & Saving](#)
 4. [Data Inspection & Info](#)
 5. [Data Selection & Filtering](#)
 6. [Data Cleaning](#)
 7. [Data Manipulation](#)
 8. [GroupBy Operations](#)
 9. [Merging & Joining](#)
 10. [Statistical Analysis](#)
 11. [Date & Time Handling](#)
 12. [Advanced Topics](#)
-

1. Pandas Introduction & Installation

কী এবং কেন?

Pandas হল Python এর একটি powerful library যা data manipulation এবং analysis এর জন্য ব্যবহৃত হয়। ML/AI প্রজেক্টে ডেটা preprocessing এর জন্য এটি অত্যাবশ্যক।

Installation

```
bash  
  
pip install pandas  
pip install numpy # Pandas dependency
```

Basic Import

```
python
```

```
import pandas as pd
import numpy as np

# Version check
print(f"Pandas version: {pd.__version__}")
```

Output:

```
Pandas version: 2.1.0
```

2. Data Structures (Series & DataFrame)

2.1 Series (1D Data)

```
python

# Creating Series
data = [10, 20, 30, 40, 50]
series = pd.Series(data)
print("Simple Series:")
print(series)

# Series with custom index
series_indexed = pd.Series(data, index=['A', 'B', 'C', 'D', 'E'])
print("\nSeries with custom index:")
print(series_indexed)

# Series from dictionary
dict_data = {'Name': 'John', 'Age': 25, 'City': 'Dhaka'}
series_dict = pd.Series(dict_data)
print("\nSeries from dictionary:")
print(series_dict)
```

Output:

Simple Series:

0 10

1 20

2 30

3 40

4 50

dtype: int64

Series with custom index:

A 10

B 20

C 30

D 40

E 50

dtype: int64

Series from dictionary:

Name John

Age 25

City Dhaka

dtype: object

2.2 DataFrame (2D Data)

python

```

# Creating DataFrame from dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'Age': [25, 30, 35, 28],
    'City': ['Dhaka', 'Chittagong', 'Sylhet', 'Rajshahi'],
    'Salary': [50000, 60000, 70000, 55000]
}

df = pd.DataFrame(data)
print("DataFrame from dictionary:")
print(df)

# Creating DataFrame from list of lists
data_list = [
    ['Alice', 25, 'Dhaka', 50000],
    ['Bob', 30, 'Chittagong', 60000],
    ['Charlie', 35, 'Sylhet', 70000]
]

df_list = pd.DataFrame(data_list, columns=['Name', 'Age', 'City', 'Salary'])
print("\nDataFrame from list of lists:")
print(df_list)

```

Output:

DataFrame from dictionary:

	Name	Age	City	Salary
0	Alice	25	Dhaka	50000
1	Bob	30	Chittagong	60000
2	Charlie	35	Sylhet	70000
3	Diana	28	Rajshahi	55000

DataFrame from list of lists:

	Name	Age	City	Salary
0	Alice	25	Dhaka	50000
1	Bob	30	Chittagong	60000
2	Charlie	35	Sylhet	70000

3. Data Loading & Saving

3.1 CSV Files

```
python

# Creating sample data for demonstration
sample_data = {
    'Student_ID': [1, 2, 3, 4, 5],
    'Name': ['Rahim', 'Karim', 'Fatima', 'Nasir', 'Salma'],
    'Math': [85, 92, 78, 88, 95],
    'English': [78, 85, 92, 80, 87],
    'Science': [90, 88, 85, 92, 89]
}

df = pd.DataFrame(sample_data)

# Save to CSV
df.to_csv('students.csv', index=False)
print("Data saved to CSV")

# Load from CSV
df_loaded = pd.read_csv('students.csv')
print("Data loaded from CSV:")
print(df_loaded)

# Reading with specific parameters
df_custom = pd.read_csv('students.csv',
                        usecols=['Name', 'Math', 'Science'], # specific columns
                        nrows=3) # first 3 rows only
print("\nCustom loading:")
print(df_custom)
```

Output:

Data saved to CSV

Data loaded from CSV:

	Student_ID	Name	Math	English	Science
0	1	Rahim	85	78	90
1	2	Karim	92	85	88
2	3	Fatima	78	92	85
3	4	Nasir	88	80	92
4	5	Salma	95	87	89

Custom loading:

	Name	Math	Science
0	Rahim	85	90
1	Karim	92	88
2	Fatima	78	85

3.2 Excel Files

python

Save to Excel

```
df.to_excel('students.xlsx', sheet_name='Grades', index=False)  
print("Data saved to Excel")
```

Load from Excel

```
df_excel = pd.read_excel('students.xlsx', sheet_name='Grades')  
print("Data loaded from Excel:")  
print(df_excel.head())
```

3.3 JSON Files

python

Save to JSON

```
df.to_json('students.json', orient='records', indent=2)
```

Load from JSON

```
df_json = pd.read_json('students.json')  
print("Data loaded from JSON:")  
print(df_json.head())
```

4. Data Inspection & Info

4.1 Basic Information

```
python

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Age': [25, 30, 35, 28, 32],
    'Salary': [50000, 60000, 70000, 55000, 65000],
    'Department': ['IT', 'HR', 'IT', 'Finance', 'HR']
})

# Basic info
print("DataFrame shape:", df.shape)
print("DataFrame size:", df.size)
print("Column names:", df.columns.tolist())
print("Index:", df.index.tolist())
print("\nData types:")
print(df.dtypes)

# Detailed info
print("\nDataFrame info:")
df.info()
```

Output:

```
DataFrame shape: (5, 4)
DataFrame size: 20
Column names: ['Name', 'Age', 'Salary', 'Department']
Index: [0, 1, 2, 3, 4]
```

Data types:

```
Name      object
Age       int64
Salary    int64
Department object
dtype: object
```

DataFrame info:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 5 entries, 0 to 4
```

```
Data columns (total 4 columns):
```

#	Column	Non-Null Count	Dtype
0	Name	5 non-null	object
1	Age	5 non-null	int64
2	Salary	5 non-null	int64
3	Department	5 non-null	object

```
dtypes: int64(2), object(2)
```

```
memory usage: 288.0+ bytes
```

4.2 Data Preview

```
python
```



```
# First and last rows
print("First 3 rows:")
print(df.head(3))

print("\nLast 2 rows:")
print(df.tail(2))

# Random sample
print("\nRandom sample:")
print(df.sample(3))

# Unique values
print("\nUnique departments:")
print(df['Department'].unique())

print("\nValue counts:")
print(df['Department'].value_counts())
```

Output:

First 3 rows:

	Name	Age	Salary	Department
0	Alice	25	50000	IT
1	Bob	30	60000	HR
2	Charlie	35	70000	IT

Last 2 rows:

	Name	Age	Salary	Department
3	Diana	28	55000	Finance
4	Eve	32	65000	HR

Random sample:

	Name	Age	Salary	Department
1	Bob	30	60000	HR
4	Eve	32	65000	HR
0	Alice	25	50000	IT

Unique departments:

['IT' 'HR' 'Finance']

Value counts:

Department

IT 2

HR 2

Finance 1

Name: count, dtype: int64

5. Data Selection & Filtering

5.1 Column Selection

python

```
# Single column
print("Names column:")
print(df['Name'])

# Multiple columns
print("\nName and Salary:")
print(df[['Name', 'Salary']])

# Using dot notation (only for valid Python identifiers)
print("\nAges using dot notation:")
print(df.Age)
```

5.2 Row Selection

```
python

# By index
print("First row:")
print(df.iloc[0])

print("\nMultiple rows by index:")
print(df.iloc[1:4])

# By label (if index has labels)
df_labeled = df.set_index('Name')
print("\nRow by label:")
print(df_labeled.loc['Alice'])
```

5.3 Conditional Filtering

```
python
```

```

# Simple condition
high_earners = df[df['Salary'] > 60000]
print("High earners (Salary > 60000):")
print(high_earners)

# Multiple conditions
it_high_earners = df[(df['Department'] == 'IT') & (df['Salary'] > 55000)]
print("\nIT high earners:")
print(it_high_earners)

# Using query method
young_employees = df.query('Age < 30')
print("\nYoung employees (Age < 30):")
print(young_employees)

# Using isin()
selected_depts = df[df['Department'].isin(['IT', 'HR'])]
print("\nIT and HR employees:")
print(selected_depts)

```

Output:

```

High earners (Salary > 60000):
   Name  Age  Salary Department
2  Charlie  35   70000         IT
4    Eve   32   65000         HR

IT high earners:
   Name  Age  Salary Department
2  Charlie  35   70000         IT

Young employees (Age < 30):
   Name  Age  Salary Department
0  Alice   25   50000         IT
3  Diana   28   55000  Finance

IT and HR employees:
   Name  Age  Salary Department
0  Alice   25   50000         IT
1   Bob   30   60000         HR
2  Charlie  35   70000         IT
4    Eve   32   65000         HR

```

6. Data Cleaning

6.1 Handling Missing Values

```
python
```

```

# Creating data with missing values
data_with_na = {
    'Name': ['Alice', 'Bob', None, 'Diana', 'Eve'],
    'Age': [25, None, 35, 28, 32],
    'Salary': [50000, 60000, 70000, None, 65000],
    'Department': ['IT', 'HR', 'IT', 'Finance', None]
}

df_na = pd.DataFrame(data_with_na)
print("Data with missing values:")
print(df_na)

# Check missing values
print("\nMissing values count:")
print(df_na.isnull().sum())

print("\nMissing values percentage:")
print((df_na.isnull().sum() / len(df_na)) * 100)

# Drop rows with any missing values
df_dropped = df_na.dropna()
print("\nAfter dropping rows with missing values:")
print(df_dropped)

# Drop columns with missing values
df_dropped_cols = df_na.dropna(axis=1)
print("\nAfter dropping columns with missing values:")
print(df_dropped_cols)

# Fill missing values
df_filled = df_na.fillna({
    'Name': 'Unknown',
    'Age': df_na['Age'].mean(),
    'Salary': df_na['Salary'].median(),
    'Department': 'Not Specified'
})
print("\nAfter filling missing values:")
print(df_filled)

```

Output:

Data with missing values:

	Name	Age	Salary	Department
0	Alice	25.0	50000.0	IT
1	Bob	NaN	60000.0	HR
2	None	35.0	70000.0	IT
3	Diana	28.0	NaN	Finance
4	Eve	32.0	65000.0	None

Missing values count:

Name	1
Age	1
Salary	1
Department	1

dtype: int64

Missing values percentage:

Name	20.0
Age	20.0
Salary	20.0
Department	20.0

dtype: float64

After dropping rows with missing values:

	Name	Age	Salary	Department
0	Alice	25.0	50000.0	IT

After dropping columns with missing values:

Empty DataFrame

Columns: []

Index: [0, 1, 2, 3, 4]

After filling missing values:

	Name	Age	Salary	Department
0	Alice	25.0	50000.0	IT
1	Bob	30.0	60000.0	HR
2	Unknown	35.0	70000.0	IT
3	Diana	28.0	62500.0	Finance
4	Eve	32.0	65000.0	Not Specified

6.2 Removing Duplicates

python

```

# Creating data with duplicates
duplicate_data = {
    'Name': ['Alice', 'Bob', 'Alice', 'Charlie', 'Bob'],
    'Age': [25, 30, 25, 35, 30],
    'Department': ['IT', 'HR', 'IT', 'IT', 'HR']
}

df_dup = pd.DataFrame(duplicate_data)
print("Data with duplicates:")
print(df_dup)

# Check for duplicates
print("\nDuplicate rows:")
print(df_dup.duplicated())

print("\nDuplicate count:", df_dup.duplicated().sum())

# Remove duplicates
df_no_dup = df_dup.drop_duplicates()
print("\nAfter removing duplicates:")
print(df_no_dup)

# Remove duplicates based on specific columns
df_no_dup_name = df_dup.drop_duplicates(subset=['Name'])
print("\nAfter removing duplicates based on Name:")
print(df_no_dup_name)

```

6.3 Data Type Conversion

```
python
```



```
# Sample data
df_types = pd.DataFrame({
    'ID': ['1', '2', '3', '4', '5'],
    'Score': ['85.5', '92.0', '78.5', '88.0', '95.5'],
    'Grade': ['A', 'A', 'B', 'A', 'A'],
    'Date': ['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04', '2024-01-05']
})

print("Original data types:")
print(df_types.dtypes)

# Convert data types
df_types['ID'] = pd.to_numeric(df_types['ID'])
df_types['Score'] = pd.to_numeric(df_types['Score'])
df_types['Date'] = pd.to_datetime(df_types['Date'])
df_types['Grade'] = df_types['Grade'].astype('category')

print("\nAfter conversion:")
print(df_types.dtypes)
print("\nConverted data:")
print(df_types)
```

7. Data Manipulation

7.1 Adding/Modifying Columns

```
python
```

```

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'Math': [85, 92, 78, 88],
    'English': [78, 85, 92, 80],
    'Science': [90, 88, 85, 92]
})

# Add new column
df['Total'] = df['Math'] + df['English'] + df['Science']
df['Average'] = df['Total'] / 3

# Conditional column
df['Grade'] = df['Average'].apply(lambda x: 'A' if x >= 90 else 'B' if x >= 80 else 'C')

# Using np.where for conditional assignment
df['Pass_Status'] = np.where(df['Average'] >= 75, 'Pass', 'Fail')

print("Data with new columns:")
print(df)

```

Output:

Data with new columns:

	Name	Math	English	Science	Total	Average	Grade	Pass_Status
0	Alice	85	78	90	253	84.333333	B	Pass
1	Bob	92	85	88	265	88.333333	B	Pass
2	Charlie	78	92	85	255	85.000000	B	Pass
3	Diana	88	80	92	260	86.666667	B	Pass

7.2 String Operations

python

```
# Sample data with text
text_data = pd.DataFrame({
    'Name': ['alice johnson', 'BOB SMITH', 'Charlie Brown', 'diana WILSON'],
    'Email': ['alice@email.com', 'bob@Email.Com', 'charlie@EMAIL.com', 'diana@email.COM']
})

# String operations
text_data['Name_Upper'] = text_data['Name'].str.upper()
text_data['Name_Title'] = text_data['Name'].str.title()
text_data['Email_Lower'] = text_data['Email'].str.lower()
text_data['First_Name'] = text_data['Name'].str.split().str[0]
text_data['Name_Length'] = text_data['Name'].str.len()

# Check if contains
text_data['Has_Brown'] = text_data['Name'].str.contains('Brown', case=False)

print("String operations:")
print(text_data)
```

7.3 Sorting

```
python
```

```
# Sample data for sorting
sort_data = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Age': [25, 30, 35, 28, 32],
    'Salary': [50000, 60000, 70000, 55000, 65000]
})

# Sort by single column
sorted_by_age = sort_data.sort_values('Age')
print("Sorted by Age:")
print(sorted_by_age)

# Sort by multiple columns
sorted_multi = sort_data.sort_values(['Age', 'Salary'], ascending=[True, False])
print("\nSorted by Age (asc) then Salary (desc):")
print(sorted_multi)

# Sort by index
sorted_by_index = sort_data.sort_index(ascending=False)
print("\nSorted by index (descending):")
print(sorted_by_index)
```

8. GroupBy Operations

8.1 Basic GroupBy

```
python
```

```
# Sample sales data
sales_data = pd.DataFrame({
    'Salesperson': ['Alice', 'Bob', 'Alice', 'Charlie', 'Bob', 'Alice', 'Charlie'],
    'Region': ['North', 'South', 'North', 'East', 'South', 'North', 'East'],
    'Product': ['A', 'B', 'A', 'C', 'B', 'C', 'A'],
    'Sales': [1000, 1500, 800, 1200, 1800, 900, 1100],
    'Quantity': [10, 15, 8, 12, 18, 9, 11]
})

print("Sales data:")
print(sales_data)

# Group by single column
grouped_by_person = sales_data.groupby('Salesperson').agg({
    'Sales': ['sum', 'mean', 'count'],
    'Quantity': 'sum'
})
print("\nSales by Salesperson:")
print(grouped_by_person)

# Group by multiple columns
grouped_by_person_region = sales_data.groupby(['Salesperson', 'Region'])['Sales'].sum()
print("\nSales by Salesperson and Region:")
print(grouped_by_person_region)
```

Output:

Sales data:

	Salesperson	Region	Product	Sales	Quantity
0	Alice	North	A	1000	10
1	Bob	South	B	1500	15
2	Alice	North	A	800	8
3	Charlie	East	C	1200	12
4	Bob	South	B	1800	18
5	Alice	North	C	900	9
6	Charlie	East	A	1100	11

Sales by Salesperson:

	Sales		Quantity	
	sum	mean	count	sum
Salesperson				
Alice	2700	900.0	3	27
Bob	3300	1650.0	2	33
Charlie	2300	1150.0	2	23

Sales by Salesperson and Region:

Salesperson	Region	
Alice	North	2700
Bob	South	3300
Charlie	East	2300

dtype: int64

8.2 Advanced GroupBy

python

```
# Custom aggregation functions
```

```
def sales_range(x):
```

```
    return x.max() - x.min()
```

```
grouped_custom = sales_data.groupby('Region').agg({
```

```
    'Sales': ['sum', 'mean', sales_range],
```

```
    'Quantity': 'mean'
```

```
}).round(2)
```

```
print("Custom aggregation by Region:")
```

```
print(grouped_custom)
```

```
# Apply custom function to groups
```

```
def top_performer(group):
```

```
    return group.loc[group['Sales'].idxmax()]
```

```
top_by_region = sales_data.groupby('Region').apply(top_performer, include_groups=False)
```

```
print("\nTop performer by Region:")
```

```
print(top_by_region)
```

9. Merging & Joining

9.1 Concatenation

```
python
```

```
# Sample DataFrames
df1 = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Score': [85, 92, 78]
})

df2 = pd.DataFrame({
    'ID': [4, 5, 6],
    'Name': ['Diana', 'Eve', 'Frank'],
    'Score': [88, 95, 82]
})

# Vertical concatenation (add rows)
df_concat = pd.concat([df1, df2], ignore_index=True)
print("Concatenated DataFrames (vertical):")
print(df_concat)

# Horizontal concatenation (add columns)
df3 = pd.DataFrame({
    'Age': [25, 30, 35],
    'City': ['Dhaka', 'Chittagong', 'Sylhet']
})

df_concat_h = pd.concat([df1, df3], axis=1)
print("\nConcatenated DataFrames (horizontal):")
print(df_concat_h)
```

9.2 Merging

```
python
```


Sample data for merging

```
employees = pd.DataFrame({  
    'emp_id': [1, 2, 3, 4, 5],  
    'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],  
    'dept_id': [101, 102, 101, 103, 102]  
})  
  
departments = pd.DataFrame({  
    'dept_id': [101, 102, 103, 104],  
    'dept_name': ['IT', 'HR', 'Finance', 'Marketing'],  
    'location': ['Dhaka', 'Chittagong', 'Sylhet', 'Rajshahi']  
})
```

```
print("Employees:")  
print(employees)  
print("\nDepartments:")  
print(departments)
```

Inner join

```
inner_merge = pd.merge(employees, departments, on='dept_id', how='inner')  
print("\nInner Join:")  
print(inner_merge)
```

Left join

```
left_merge = pd.merge(employees, departments, on='dept_id', how='left')  
print("\nLeft Join:")  
print(left_merge)
```

Right join

```
right_merge = pd.merge(employees, departments, on='dept_id', how='right')  
print("\nRight Join:")  
print(right_merge)
```

Outer join

```
outer_merge = pd.merge(employees, departments, on='dept_id', how='outer')  
print("\nOuter Join:")  
print(outer_merge)
```

10. Statistical Analysis

10.1 Descriptive Statistics

```
python
```

```
# Sample numerical data
```

```
numeric_data = pd.DataFrame({  
    'Height': [165, 170, 175, 168, 172, 169, 171, 174, 167, 173],  
    'Weight': [55, 65, 70, 60, 68, 62, 66, 72, 58, 69],  
    'Age': [25, 30, 35, 28, 32, 29, 31, 34, 26, 33]  
})
```

```
# Basic statistics
```

```
print("Descriptive Statistics:")  
print(numeric_data.describe())
```

```
# Individual statistics
```

```
print(f"\nMean Heights: {numeric_data['Height'].mean():.2f}")  
print(f"Median Weight: {numeric_data['Weight'].median():.2f}")  
print(f"Standard deviation of Age: {numeric_data['Age'].std():.2f}")  
print(f"Correlation between Height and Weight: {numeric_data['Height'].corr(numeric_data['Weight']):.3f}")
```

```
# Correlation matrix
```

```
print("\nCorrelation Matrix:")  
print(numeric_data.corr().round(3))
```

Output:

Descriptive Statistics:

	Height	Weight	Age
count	10.000000	10.000000	10.000000
mean	170.400000	63.500000	30.300000
std	3.373479	5.560276	3.683317
min	165.000000	55.000000	25.000000
25%	168.250000	59.500000	27.750000
50%	170.500000	63.500000	30.500000
75%	172.750000	68.250000	32.500000
max	175.000000	72.000000	35.000000

Mean Heights: 170.40

Median Weight: 63.50

Standard deviation of Age: 3.68

Correlation between Height and Weight: 0.842

Correlation Matrix:

	Height	Weight	Age
Height	1.000	0.842	0.688
Weight	0.842	1.000	0.736
Age	0.688	0.736	1.000

10.2 Value Counts and Cross-tabulation

python

Categorical data

```
categorical_data = pd.DataFrame({  
    'Gender': ['M', 'F', 'M', 'F', 'M', 'F', 'M', 'F', 'M', 'F'],  
    'Department': ['IT', 'HR', 'IT', 'Finance', 'HR', 'IT', 'Finance', 'HR', 'IT', 'Finance'],  
    'Experience': ['Junior', 'Senior', 'Senior', 'Junior', 'Senior', 'Junior', 'Senior', 'Senior', 'Junior', 'Senior']  
})
```

Value counts

```
print("Department distribution:")  
print(categorical_data['Department'].value_counts())
```

Cross-tabulation

```
print("\nCross-tabulation (Gender vs Department):")  
crosstab = pd.crosstab(categorical_data['Gender'], categorical_data['Department'])  
print(crosstab)
```

Percentage cross-tabulation

```
print("\nPercentage cross-tabulation:")  
print(pd.crosstab(categorical_data['Gender'], categorical_data['Department'], normalize='all') * 100)
```

11. Date & Time Handling

11.1 Creating and Parsing Dates

```
python
```

```
# Creating date data
```

```
date_strings = ['2024-01-15', '2024-02-20', '2024-03-10', '2024-04-05', '2024-05-12']
```

```
dates = pd.to_datetime(date_strings)
```

```
date_data = pd.DataFrame({
```

```
    'Date': dates,
```

```
    'Sales': [1000, 1200, 800, 1500, 1100],
```

```
    'Customers': [50, 60, 40, 75, 55]
```

```
})
```

```
print("Date data:")
```

```
print(date_data)
```

```
# Extract date components
```

```
date_data['Year'] = date_data['Date'].dt.year
```

```
date_data['Month'] = date_data['Date'].dt.month
```

```
date_data['Day'] = date_data['Date'].dt.day
```

```
date_data['Weekday'] = date_data['Date'].dt.day_name()
```

```
print("\nWith extracted date components:")
```

```
print(date_data)
```

11.2 Date Range and Resampling

```
python
```

```
# Create date range
date_range = pd.date_range(start='2024-01-01', end='2024-01-31', freq='D')
daily_data = pd.DataFrame({
    'Date': date_range,
    'Sales': np.random.randint(100, 1000, len(date_range))
})

# Set date as index
daily_data.set_index('Date', inplace=True)
print("Daily sales data (first 10 rows):")
print(daily_data.head(10))

# Resample to weekly
weekly_sales = daily_data.resample("W").sum()
print("\nWeekly sales:")
print(weekly_sales)

# Resample to monthly
monthly_avg = daily_data.resample('M').mean()
print("\nMonthly average:")
print(monthly_avg)
```

12. Advanced Topics

12.1 Pivot Tables

```
python
```

```

# Sample data for pivot table
pivot_data = pd.DataFrame({
    'Date': pd.date_range('2024-01-01', periods=12, freq='M'),
    'Region': ['North', 'South', 'East', 'West'] * 3,
    'Product': ['A', 'B', 'A', 'B', 'A', 'B', 'A', 'B', 'A', 'B', 'A', 'B'],
    'Sales': [1000, 1200, 800, 1500, 1100, 1300, 900, 1400, 1050, 1250, 950, 1350],
    'Quantity': [100, 120, 80, 150, 110, 130, 90, 140, 105, 125, 95, 135]
})

print("Pivot data:")
print(pivot_data.head())

# Create pivot table
pivot_table = pd.pivot_table(pivot_data,
                              values='Sales',
                              index='Region',
                              columns='Product',
                              aggfunc='mean')

print("\nPivot table (Average Sales by Region and Product):")
print(pivot_table)

# Multiple aggregation functions
multi_pivot = pd.pivot_table(pivot_data,
                              values=['Sales', 'Quantity'],
                              index='Region',
                              columns='Product',
                              aggfunc={'Sales': 'sum', 'Quantity': 'mean'})

print("\nMulti-function pivot table:")
print(multi_pivot)

```

Output:

Pivot data:

	Date	Region	Product	Sales	Quantity
0	2024-01-31	North	A	1000	100
1	2024-02-29	South	B	1200	120
2	2024-03-31	East	A	800	80
3	2024-04-30	West	B	1500	150
4	2024-05-31	North	A	1100	110

Pivot table (Average Sales by Region and Product):

Product	A	B
Region		
East	875.0	1400.0
North	1050.0	1300.0
South	950.0	1275.0
West	1350.0	1450.0

Multi-function pivot table:

	Quantity		Sales	
Product	A	B	A	B
Region				
East	82.5	140.0	1750	2800
North	105.0	130.0	2100	2600
South	95.0	127.5	1900	2550
West	135.0	145.0	2700	2900

12.2 Multi-level Indexing

python


```

# Create multi-index DataFrame
arrays = [
    ['A', 'A', 'B', 'B'],
    ['X', 'Y', 'X', 'Y']
]
multi_index = pd.MultiIndex.from_arrays(arrays, names=['Group', 'Subgroup'])

multi_df = pd.DataFrame({
    'Value1': [10, 20, 30, 40],
    'Value2': [100, 200, 300, 400]
}, index=multi_index)

print("Multi-index DataFrame:")
print(multi_df)

# Access data with multi-index
print("\nAccessing Group A:")
print(multi_df.loc['A'])

print("\nAccessing specific combination:")
print(multi_df.loc[('A', 'X')])

# Reset index
reset_df = multi_df.reset_index()
print("\nAfter resetting index:")
print(reset_df)

# Set multi-index from columns
multi_from_cols = reset_df.set_index(['Group', 'Subgroup'])
print("\nMulti-index from columns:")
print(multi_from_cols)

```

12.3 Window Functions (Rolling & Expanding)

```
python
```

```

# Time series data
ts_data = pd.DataFrame({
    'Date': pd.date_range('2024-01-01', periods=20, freq='D'),
    'Price': [100, 102, 98, 105, 107, 103, 110, 108, 112, 115,
              118, 116, 120, 122, 119, 125, 128, 130, 127, 132]
})
ts_data.set_index('Date', inplace=True)

# Rolling window calculations
ts_data['MA_3'] = ts_data['Price'].rolling(window=3).mean() # 3-day moving average
ts_data['MA_7'] = ts_data['Price'].rolling(window=7).mean() # 7-day moving average
ts_data['Rolling_Std'] = ts_data['Price'].rolling(window=5).std() # 5-day rolling std

# Expanding window calculations
ts_data['Expanding_Mean'] = ts_data['Price'].expanding().mean()
ts_data['Expanding_Max'] = ts_data['Price'].expanding().max()

print("Time series with rolling and expanding windows:")
print(ts_data.round(2))

# Percentage change
ts_data['Daily_Return'] = ts_data['Price'].pct_change()
ts_data['Cumulative_Return'] = (1 + ts_data['Daily_Return']).cumprod() - 1

print("\nWith returns calculated:")
print(ts_data[['Price', 'Daily_Return', 'Cumulative_Return']].round(4))

```

12.4 Data Transformation & Mapping

```
python
```

```

# Sample data for transformation
transform_data = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Score': [85, 92, 78, 88, 95],
    'Grade_Letter': ['B', 'A', 'C', 'B', 'A']
})

# Apply function to transform data
def categorize_score(score):
    if score >= 90:
        return 'Excellent'
    elif score >= 80:
        return 'Good'
    elif score >= 70:
        return 'Average'
    else:
        return 'Poor'

transform_data['Performance'] = transform_data['Score'].apply(categorize_score)

# Map values
grade_mapping = {'A': 4.0, 'B': 3.0, 'C': 2.0, 'D': 1.0, 'F': 0.0}
transform_data['GPA'] = transform_data['Grade_Letter'].map(grade_mapping)

# Lambda functions
transform_data['Score_Squared'] = transform_data['Score'].apply(lambda x: x**2)
transform_data['Name_Length'] = transform_data['Name'].apply(len)

print("Transformed data:")
print(transform_data)

# Group transformation
transform_data['Score_Rank'] = transform_data['Score'].rank(ascending=False)
transform_data['Score_Percentile'] = transform_data['Score'].rank(pct=True)

print("\nWith rankings:")
print(transform_data.round(2))

```

12.5 Working with Categories

python

```

# Categorical data
cat_data = pd.DataFrame({
    'Size': ['Small', 'Medium', 'Large', 'Small', 'Large', 'Medium', 'Small'],
    'Priority': ['High', 'Low', 'Medium', 'High', 'Low', 'Medium', 'High'],
    'Status': ['Active', 'Inactive', 'Active', 'Pending', 'Active', 'Inactive', 'Pending']
})

# Convert to categorical
cat_data['Size'] = pd.Categorical(cat_data['Size'],
                                categories=['Small', 'Medium', 'Large'],
                                ordered=True)

cat_data['Priority'] = pd.Categorical(cat_data['Priority'],
                                    categories=['Low', 'Medium', 'High'],
                                    ordered=True)

print("Categorical data:")
print(cat_data)
print("\nData types:")
print(cat_data.dtypes)

# Categorical operations
print("\nSize value counts (ordered):")
print(cat_data['Size'].value_counts().sort_index())

# Sort by categorical order
sorted_cat = cat_data.sort_values(['Priority', 'Size'], ascending=[False, True])
print("\nSorted by Priority (desc) and Size (asc):")
print(sorted_cat)

# Add new category
cat_data['Size'] = cat_data['Size'].cat.add_categories(['Extra Large'])
print("\nCategories after adding 'Extra Large':")
print(cat_data['Size'].cat.categories)

```

12.6 Performance Tips & Memory Optimization

python

```

# Memory usage analysis
sample_data = pd.DataFrame({
    'ID': range(10000),
    'Category': ['A', 'B', 'C', 'D', 'E'] * 2000,
    'Value': np.random.randn(10000),
    'Status': ['Active', 'Inactive'] * 5000
})

print("Memory usage before optimization:")
print(sample_data.info(memory_usage='deep'))

# Optimize data types
sample_data['ID'] = sample_data['ID'].astype('int32') # Instead of int64
sample_data['Category'] = sample_data['Category'].astype('category')
sample_data['Status'] = sample_data['Status'].astype('category')
sample_data['Value'] = sample_data['Value'].astype('float32') # Instead of float64

print("\nMemory usage after optimization:")
print(sample_data.info(memory_usage='deep'))

# Chunk processing for large files
def process_large_file(filename, chunksize=1000):
    """Process large CSV file in chunks"""
    chunk_results = []

    for chunk in pd.read_csv(filename, chunksize=chunksize):
        # Process each chunk
        processed_chunk = chunk.groupby('Category').sum()
        chunk_results.append(processed_chunk)

    # Combine all chunks
    final_result = pd.concat(chunk_results).groupby(level=0).sum()
    return final_result

# Example usage (commented as file may not exist)
# result = process_large_file('large_data.csv', chunksize=5000)

```

ML/AI ଏଠାରେ Pandas ଏବଂ Practical Applications

Data Preprocessing Pipeline

python

```

def preprocess_ml_data(df):
    """Complete preprocessing pipeline for ML """

    # 1. Handle missing values
    numeric_columns = df.select_dtypes(include=[np.number]).columns
    categorical_columns = df.select_dtypes(include=['object']).columns

    # Fill numeric missing values with median
    df[numeric_columns] = df[numeric_columns].fillna(df[numeric_columns].median())

    # Fill categorical missing values with mode
    for col in categorical_columns:
        df[col] = df[col].fillna(df[col].mode()[0])

    # 2. Remove duplicates
    df = df.drop_duplicates()

    # 3. Handle outliers (using IQR method)
    for col in numeric_columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]

    # 4. Encode categorical variables
    df_encoded = pd.get_dummies(df, columns=categorical_columns, drop_first=True)

    # 5. Feature scaling (normalization)
    from sklearn.preprocessing import StandardScaler
    scaler = StandardScaler()
    numeric_columns_encoded = df_encoded.select_dtypes(include=[np.number]).columns
    df_encoded[numeric_columns_encoded] = scaler.fit_transform(df_encoded[numeric_columns_encoded])

    return df_encoded, scaler

# Example usage
ml_data = pd.DataFrame({
    'Age': [25, 30, 35, 28, 32, np.nan, 29],
    'Income': [50000, 60000, 70000, 55000, 65000, 58000, np.nan],
    'Education': ['Bachelor', 'Master', 'PhD', 'Bachelor', 'Master', 'Bachelor', 'Master'],
    'Target': [0, 1, 1, 0, 1, 0, 1]
})

```

```
})

print("Original ML data:")
print(ml_data)

# Apply preprocessing
processed_data, scaler = preprocess_ml_data(ml_data.drop('Target', axis=1))
print("\nProcessed data for ML:")
print(processed_data)
```

Key Points for ML/AI:

1. Data Loading & Exploration

- বিভিন্ন format এর data load করা (CSV, Excel, JSON, SQL)
- Data এর shape, size, types বোঝা
- Missing values identify করা

2. Data Cleaning

- Missing values handle করা
- Duplicates remove করা
- Outliers detect ও handle করা

3. Feature Engineering

- নতুন features create করা
- Categorical encoding (Label encoding, One-hot encoding)
- Date/time features extract করা

4. Data Transformation

- Scaling/Normalization
- Log transformation
- Binning/Discretization

5. Statistical Analysis

- Descriptive statistics
- Correlation analysis
- Distribution analysis

6. Performance Optimization

- Memory efficient data types
- Vectorized operations
- Chunk processing for large datasets

Practice Exercises:

Exercise 1: Student Performance Analysis

```
python

# Create student performance dataset and analyze
students = pd.DataFrame({
    'Student_ID': range(1, 101),
    'Name': [f'Student_{i}' for i in range(1, 101)],
    'Math': np.random.normal(75, 15, 100),
    'Science': np.random.normal(80, 12, 100),
    'English': np.random.normal(78, 10, 100),
    'Attendance': np.random.uniform(70, 100, 100)
})

# Tasks:
# 1. Calculate total and average scores
# 2. Assign grades based on average
# 3. Find top 10 students
# 4. Analyze correlation between subjects
# 5. Create performance categories
```

Exercise 2: Sales Data Analysis

```
python
```



```
# Create sales dataset and perform analysis
np.random.seed(42)
dates = pd.date_range('2023-01-01', '2023-12-31', freq='D')
sales = pd.DataFrame({
    'Date': dates,
    'Product': np.random.choice(['A', 'B', 'C', 'D'], len(dates)),
    'Region': np.random.choice(['North', 'South', 'East', 'West'], len(dates)),
    'Sales': np.random.normal(1000, 200, len(dates)),
    'Quantity': np.random.poisson(50, len(dates))
})
```

```
# Tasks:
# 1. Monthly sales trends
# 2. Best performing products and regions
# 3. Seasonal patterns
# 4. Moving averages
# 5. Year-over-year growth (if multi-year data)
```

Summary

এই complete Pandas course এ আপনি শিখেছেন:

1. **Basic Data Structures:** Series ও DataFrame
2. **Data I/O:** বিভিন্ন format এর data load/save
3. **Data Inspection:** Data explore করার techniques
4. **Data Selection:** Filtering ও selection methods
5. **Data Cleaning:** Missing values, duplicates handle
6. **Data Manipulation:** Transform, sort, modify
7. **GroupBy:** Group-wise operations
8. **Merging:** Data combination techniques
9. **Statistics:** Descriptive ও analytical functions
10. **Date/Time:** Time series operations
11. **Advanced Topics:** Pivot tables, multi-indexing, window functions
12. **ML Applications:** ML এর জন্য data preprocessing

এই সব techniques ML এবং AI projects এ data preprocessing এর জন্য essential। প্রতিটি example hands-on practice করুন এবং নিজের dataset দিয়ে experiment করুন।